EECS 348 Assignment 5: GenAI Analysis

Nicolas Villalobos

October 29, 2026

Overview

I modified Christopher Warren's Assignment 4 Sudoku solver to read `puzzle1.txt`–`puzzle5.txt`, write `solutionN.txt`, and be object-oriented. I gave the same prompt to two GenAIs and compared their outputs.

1 GenAI Programs Used (a)

1. **Mistral AI**, using its fast mode on the most recent model available in Le Chat.
2. **Claude Opus 5.5**, developed by Anthropic.

2 How I Accessed Them (b)

- **Mistral AI:** Le Chat in my web browser (`https://chat.mistral.ai`), fast mode, in a new chat.
- **Claude Opus 5.5:** claude.ai in my web browser (`https://claude.ai`), in a new chat with no earlier context.

In both cases I attached Christopher's original `CW_SudokuSolver.cpp` along with the prompt below.

3 Prompt Used (c)

The identical prompt was given to both GenAIs:

```
Modify this C++ Sudoku solver so that it reads puzzle1.txt through puzzle5.txt and
writes all solutions for puzzle N to solutionN.txt in the same format as the input,
followed by the line "Solutions found:  n", or the single line "No solution found."
if there are none.  Make the solution object-oriented and keep the recursive backtracking.
```

4 Generated Code (d)

The complete, unedited code from each GenAI follows.

4.1 Mistral AI

```cpp
//Name of program: CW_SudokuSolver.cpp
//Description: An object-oriented C++ program that reads puzzle1.txt through puzzle5.txt,
//each containing an unfinished 9x9 sudoku board, and writes every possible solution
//for puzzle N to solutionN.txt (same grid format as the input), followed by the line
//"Solutions found: n", or the single line "No solution found." if there are none.
//Solving uses recursive backtracking.
//
//Inputs: puzzle1.txt ... puzzle5.txt, nine lines of nine characters
//(digits 1-9, or '_' for an empty square).
//
//Outputs: solution1.txt ... solution5.txt.
//
//Collaborators: none
//Other sources: Google Gemini's code, ChatGPT's code for pros and cons,
//as well as some improvements
//Author: Christopher Warren
//Creation date: 10/15/2026
//Revision date: 10/29/2026
//Revisions: 1. added comments 2. included validity checker.
//3. implemented while loop to skip unnecessary recursions.
//4. restructured solveSudoku function to find empty spaces before recursing.
//5. restructured into a SudokuSolver class and batch mode (puzzle1-5 -> solution1-5).

#include <iostream> // input/output
#include <fstream> // reading/writing files
#include <sstream> // string streams (building output)
#include <vector> // vectors (for board)
#include <string> // strings

const int SIZE = 9; // our sudoku board will always be 9x9, so we keep it constant here

// Class SudokuSolver
// Holds one puzzle board, its solution count, and an output string stream.
// All solving/printing logic lives in this class so each puzzle instance
// is independent. The core algorithm is the same recursive backtracking
// as before: try each digit 1-9 in the current empty cell, recurse to the
// next empty cell, and backtrack on return.
class SudokuSolver {
private:
    std::vector<std::vector<char>> board; // the 9x9 grid ('1'-'9' or '_')
    int solutionCount; // number of solutions found so far
    std::ostringstream output; // accumulated solutions, ready to write to file

    // Private helper: findNextEmpty
    // Scans the board (row-major) starting just after (row, col) for the next
    // empty square. Returns true and sets nextRow/nextCol if one is found.
    // Inputs: starting row and column, output row/column by reference
    // Output: true if an empty square exists, false if the board is full
    bool findNextEmpty(int row, int col, int& nextRow, int& nextCol) const {
        nextRow = row;
        nextCol = col + 1;
        while (nextRow < SIZE) { // while we aren't at the end of the board
            if (nextCol == SIZE) { // if we hit the end of a row
                nextRow = nextRow + 1; // move to the next row
                nextCol = 0; // first item in the row
                continue; // skip the rest
            }
            if (board[nextRow][nextCol] == '_') { // if we find an empty space
                return true; // report it
```

```
            }
            nextCol = nextCol + 1; // next index in row
        }
        return false; // board is complete
    }

    // Private method: isValid
    // Checks whether placing digit val at board[row][col] breaks any rule:
    // 1. no same value already in the same row
    // 2. no same value already in the same column
    // 3. no same value already in the same 3x3 grid
    // Inputs: row, col, and the digit val
    // Output: true if the placement is valid, false otherwise
    bool isValid(int row, int col, char val) const {
        // check the full length/width of the board for row/column conflicts
        for (int i = 0; i < SIZE; ++i) {
            if (board[row][i] == val) return false; // row conflict
            if (board[i][col] == val) return false; // column conflict
        }

        // integer division by 3 rounds down, so multiplying by 3 gives
        // 0, 3, or 6 - the starting row/column of the local 3x3 grid
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // only check the local 3x3 grid for conflicts
        for (int r = startRow; r < startRow + 3; ++r) {
            for (int c = startCol; c < startCol + 3; ++c) {
                if (board[r][c] == val) return false; // grid conflict
            }
        }
        return true; // no conflicts, the digit is valid
    }

    // Private method: formatBoard
    // Returns the board as text in the same format as the input:
    // nine lines of nine space-separated characters.
    std::string formatBoard() const {
        std::ostringstream text;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                text << board[r][c] << (c == SIZE - 1 ? "" : " ");
            }
            text << "\n";
        }
        return text.str();
    }

    // Private method: solveFrom
    // The recursive backtracking core. Tries digits '1'-'9' at the current
    // empty cell; for each valid one, find the next empty cell and recurse.
    // If no empty cell remains, the board is a solution: record and count it.
    // After each recursive call, undo the placement (backtrack).
    // Inputs: the current empty cell's row and column
    // Output: none (solutions recorded in output/solutionCount)
    void solveFrom(int row, int col) {
        for (char val = '1'; val <= '9'; ++val) { // try each candidate digit
            if (isValid(row, col, val)) {
                board[row][col] = val; // tentatively place value

                int nextRow, nextCol;
                if (findNextEmpty(row, col, nextRow, nextCol)) {
                    solveFrom(nextRow, nextCol); // recurse into next empty cell
                } else {
                    // board is full: this placement completes a solution
                    solutionCount++;
                    output << formatBoard() << "\n"; // grids separated by a blank line
                }
```

```
128                 board[row][col] = '_'; // backtrack
129             }
130         }
131     }
132
133 public:
134     // Constructor: builds a solver from a 9x9 grid of characters
135     explicit SudokuSolver(const std::vector<std::vector<char>>& grid)
136         : board(grid), solutionCount(0) {}
137
138     // Public method: isBoardValid
139     // Checks the initial board for rule violations among the given clues.
140     // Temporarily blanks each clue and tests it against the rest of the
141     // board, restoring it afterwards. Saves enormous search time on
142     // unsolvable boards.
143     // Output: true if every given digit is legal, false otherwise
144     bool isBoardValid() {
145         char temp = '_';
146         for (int row = 0; row < SIZE; ++row) {
147             for (int col = 0; col < SIZE; ++col) {
148                 if (board[row][col] != '_') { // a given clue
149                     temp = board[row][col]; // save its value
150                     board[row][col] = '_'; // blank it out
151                     if (!isValid(row, col, temp)) { // illegal among the rest?
152                         board[row][col] = temp; // restore before failing
153                         return false; // board has no solutions
154                     }
155                     board[row][col] = temp; // restore and continue
156                 }
157             }
158         }
159         return true; // all given digits are legal
160     }
161
162     // Public method: solve
163     // Validates the board, then runs recursive backtracking from the first
164     // empty cell. Records every solution.
165     // Output: none (results kept internally)
166     void solve() {
167         if (!isBoardValid()) return; // invalid starting clues: nothing to do
168
169         for (int r = 0; r < SIZE; ++r) { // find first empty cell
170             for (int c = 0; c < SIZE; ++c) {
171                 if (board[r][c] == '_') {
172                     solveFrom(r, c); // start backtracking here
173                     return;
174                 }
175             }
176         }
177         // no empty cells: the board was already a complete solution
178         solutionCount = 1;
179         output << formatBoard() << "\n";
180     }
181
182     // Public method: results
183     // Returns the text to write to the solution file:
184     // all solutions (if any) followed by "Solutions found: n",
185     // or the single line "No solution found." if there were none.
186     std::string results() const {
187         std::ostringstream text;
188         if (solutionCount == 0) {
189             text << "No solution found.\n";
190         } else {
191             text << output.str();
192             text << "Solutions found: " << solutionCount << "\n";
193         }
194         return text.str();
195     }
```

```
196
197     // Public method: count
198     // Number of solutions found by solve().
199     int count() const { return solutionCount; }
200 };
201
202 // Function loadBoard
203 // Reads a 9x9 grid of characters from a file stream into a board matrix.
204 // Inputs: an open ifstream
205 // Outputs: true and fills the grid on success, false on bad format
206 bool loadBoard(std::ifstream& infile, std::vector<std::vector<char>>& grid) {
207     grid.assign(SIZE, std::vector<char>(SIZE));
208     for (int r = 0; r < SIZE; ++r) {
209         for (int c = 0; c < SIZE; ++c) {
210             if (!(infile >> grid[r][c])) { // fewer characters than expected
211                 return false; // invalid file format
212             }
213         }
214     }
215     return true;
216 }
217
218 // main function
219 // Runs the solver over puzzle1.txt .. puzzle5.txt. For each puzzle N that
220 // exists, writes all solutions (or "No solution found.") to solutionN.txt.
221 int main() {
222     const int NUM_PUZZLES = 5;
223     bool anyWorked = false;
224
225     for (int n = 1; n <= NUM_PUZZLES; ++n) {
226         std::string inName = "puzzle" + std::to_string(n) + ".txt";
227         std::string outName = "solution" + std::to_string(n) + ".txt";
228
229         std::ifstream infile(inName);
230         if (!infile) { // missing puzzle file: skip with a notice, don't abort the batch
231             std::cerr << "Skipped " << inName << ": could not open file.\n";
232             continue;
233         }
234
235         std::vector<std::vector<char>> grid;
236         if (!loadBoard(infile, grid)) {
237             std::cerr << "Skipped " << inName << ": invalid file format. Expected a 9x9 grid.\n";
238             continue;
239         }
240         infile.close();
241
242         SudokuSolver solver(grid); // one object per puzzle
243         solver.solve(); // recursive backtracking happens here
244
245         std::ofstream outfile(outName);
246         if (!outfile) {
247             std::cerr << "Error: could not write " << outName << "\n";
248             continue;
249         }
250         outfile << solver.results();
251         outfile.close();
252
253         std::cout << inName << " -> " << outName << ": "
254                   << solver.count() << " solution(s).\n";
255         anyWorked = true;
256     }
257
258     if (!anyWorked) {
259         std::cerr << "Error: no puzzle files could be processed.\n";
260         return 1;
261     }
262     return 0;
263 }
```

4.2 Claude Opus 5.5

```cpp
// Name of program: CW_SudokuSolver.cpp
// Description: An object-oriented C++ program that reads unfinished Sudoku boards
// from puzzle1.txt through puzzle5.txt and uses recursive backtracking to find
// every possible solution to each one.
//
// Inputs: Each puzzle file should contain nine lines of nine characters,
// where each character is a digit (1-9) or an underscore (_) for an empty square.
//
// Outputs: For puzzle N, every solution is written to solutionN.txt in the same
// format as the input (solutions separated by a blank line), followed by the line
// "Solutions found: n". If there are no solutions, the file contains the single
// line "No solution found."
//
// Collaborators: none
// Other sources: Google Gemini's code, ChatGPT's code for pros and cons
// as well as some improvements
// Author: Christopher Warren
// Creation date: 10/15/2026
// Revision date: 10/29/2026
// Revisions: 1. added comments 2. included validity checker
// 3. implemented while loop to skip unnecessary recursions
// 4. restructured solveSudoku function to find empty spaces before calling recursive function
// 5. restructured into SudokuBoard and SudokuSolver classes
// 6. reads puzzle1.txt - puzzle5.txt and writes results to solution1.txt - solution5.txt

#include <iostream> // input/output
#include <fstream> // reading/writing files
#include <vector> // vectors (for board)
#include <string> // strings

// Class SudokuBoard
// Stores a 9x9 grid and knows how to load itself, check placements,
// and write itself to a stream.
class SudokuBoard {
public:
    static const int SIZE = 9; // board is always 9x9
    static const char EMPTY = '_'; // character for an empty square

    // constructor: starts with an all-empty board
    SudokuBoard() : grid(SIZE, std::vector<char>(SIZE, EMPTY)) {}

    // loadFromFile
    // reads 81 characters from the file into the grid
    // inputs: filename
    // outputs: true if the file opened and had a valid 9x9 grid, false otherwise
    bool loadFromFile(const std::string& filename) {
        std::ifstream infile(filename);
        if (!infile) { // file could not be opened
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                char ch;
                if (!(infile >> ch)) { // fewer characters than expected
                    std::cerr << "Error: " << filename << " is not a 9x9 grid.\n";
                    return false;
                }
                if (ch != EMPTY && (ch < '1' || ch > '9')) { // bad character
                    std::cerr << "Error: " << filename << " contains invalid character '"
                              << ch << "'.\n";
                    return false;
                }
                grid[r][c] = ch;
            }
```

```
        }
        return true;
    }

    // get / set / clear a single square
    char get(int row, int col) const { return grid[row][col]; }
    void set(int row, int col, char val) { grid[row][col] = val; }
    void clear(int row, int col) { grid[row][col] = EMPTY; }
    bool isEmpty(int row, int col) const { return grid[row][col] == EMPTY; }

    // canPlace
    // checks if val could go at (row, col) without repeating in the
    // same row, column, or 3x3 box. The square itself is ignored,
    // so this also works for checking digits already on the board.
    // inputs: row, column, digit
    // outputs: true if valid, false if it conflicts
    bool canPlace(int row, int col, char val) const {
        for (int i = 0; i < SIZE; ++i) {
            if (i != col && grid[row][i] == val) return false; // row check
            if (i != row && grid[i][col] == val) return false; // column check
        }
        // row / 3 rounds down, so * 3 gives the box start: 0, 3, or 6
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int r = startRow; r < startRow + 3; ++r) {
            for (int c = startCol; c < startCol + 3; ++c) {
                if ((r != row || c != col) && grid[r][c] == val) return false; // box check
            }
        }
        return true;
    }

    // isConsistent
    // checks that the starting digits don't already break the rules.
    // If they do, there are no solutions and we can skip solving entirely.
    // outputs: true if every given digit is valid, false otherwise
    bool isConsistent() const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!isEmpty(r, c) && !canPlace(r, c, grid[r][c])) return false;
            }
        }
        return true;
    }

    // findEmpty
    // starting at (row, col) and moving left to right, top to bottom,
    // finds the next empty square
    // inputs: row and col by reference (updated to the empty square's position)
    // outputs: true if an empty square was found, false if the board is full
    bool findEmpty(int& row, int& col) const {
        while (row < SIZE) {
            if (col == SIZE) { // end of row, move to the next one
                ++row;
                col = 0;
                continue;
            }
            if (isEmpty(row, col)) return true;
            ++col;
        }
        return false;
    }

    // print
    // writes the board in the same format as the input file
    // inputs: the output stream (file or terminal)
    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
```

```
            for (int c = 0; c < SIZE; ++c) {
                out << grid[r][c];
            }
            out << "\n";
        }
    }

private:
    std::vector<std::vector<char>> grid; // the 9x9 board
};

// Class SudokuSolver
// Uses recursive backtracking to find every solution of a board
// and writes each one to an output stream as it is found.
class SudokuSolver {
public:
    // constructor: takes the board to solve and where solutions go
    SudokuSolver(const SudokuBoard& start, std::ostream& output)
        : board(start), out(output), solutionCount(0) {}

    // solveAll
    // finds and writes every solution
    // outputs: the number of solutions found
    int solveAll() {
        solutionCount = 0;
        if (!board.isConsistent()) return 0; // unsolvable from the start

        int row = 0, col = 0;
        if (board.findEmpty(row, col)) {
            solve(row, col); // start recursion at the first empty square
        } else {
            recordSolution(); // board was already full and valid
        }
        return solutionCount;
    }

private:
    SudokuBoard board; // working copy that gets filled in
    std::ostream& out; // where solutions are written
    int solutionCount; // total solutions found

    // solve (recursive)
    // tries digits 1-9 in the empty square at (row, col). For each valid
    // digit, places it, finds the next empty square, and recurses. If no
    // empty square is left, the board is a solution. Afterward the square
    // is cleared (backtrack) so the next digit can be tried.
    // inputs: row and column of an empty square
    void solve(int row, int col) {
        for (char val = '1'; val <= '9'; ++val) {
            if (board.canPlace(row, col, val)) {
                board.set(row, col, val); // tentatively place value

                int nextRow = row;
                int nextCol = col + 1;
                if (board.findEmpty(nextRow, nextCol)) {
                    solve(nextRow, nextCol); // recurse to next empty square
                } else {
                    recordSolution(); // board is full: a solution
                }

                board.clear(row, col); // backtrack
            }
        }
    }

    // recordSolution
    // writes the current full board to the output, with a blank line
    // between solutions, and increases the count
```

```
    void recordSolution() {
        if (solutionCount > 0) out << "\n";
        board.print(out);
        ++solutionCount;
    }
};

// main
// for N = 1 to 5, reads puzzleN.txt, solves it, and writes the
// results to solutionN.txt
int main() {
    const int NUM_PUZZLES = 5;

    for (int n = 1; n <= NUM_PUZZLES; ++n) {
        std::string inName = "puzzle" + std::to_string(n) + ".txt";
        std::string outName = "solution" + std::to_string(n) + ".txt";

        SudokuBoard board;
        if (!board.loadFromFile(inName)) { // skip puzzles that can't be read
            std::cerr << "Skipping " << inName << "\n";
            continue;
        }

        std::ofstream outfile(outName);
        if (!outfile) {
            std::cerr << "Error: Could not create " << outName << "\n";
            continue;
        }

        SudokuSolver solver(board, outfile);
        int count = solver.solveAll();

        if (count == 0) {
            outfile << "No solution found.\n";
        } else {
            outfile << "Solutions found: " << count << "\n";
        }

        std::cout << inName << " -> " << outName << " (" << count << " solution"
                  << (count == 1 ? "" : "s") << ")\n";
    }

    return 0;
}
```

5 Analysis (e)

Both outputs were compiled with `g++ -std=c++11 -Wall -Wextra` and run on all five puzzles. The correct counts, confirmed with an independent solver, are 1, 1, 1, 0, and 2.

Correctness. Mistral compiles in every configuration and produces all five files correctly, but it accepts invalid characters like `0` or `x`. Claude fails to link at `-O0`: `static const char EMPTY` is declared in the class but never defined (`undefined reference to SudokuBoard::EMPTY`). It also prints boards with no spaces (`539824176`), so 4 of 5 files have the wrong format.

Execution time. Both use the original recursive backtracking, so both are $O(9^e)$ for e empty squares, with an $O(27)$ check per placement. Claude adds a few extra comparisons per check. Both averaged about 4 ms over 200 runs, so there is no meaningful difference.

Space complexity. Both store one 9×9 board and use an $O(e)$ recursion stack. Mistral buffers all k solutions in an `ostringstream` ($O(k)$); Claude writes each one to the file as it is found ($O(1)$).

Maintainability. Mistral uses one class with detailed comments. Claude splits the board and solver into two classes, which is cleaner, but comments less. Both rewrote the entire file and dropped a "Revised by" line, so neither can be submitted as-is.

Criterion	Mistral AI	Claude Opus 5.5	Final
Compiles at `-O0`	Yes	No	Yes
Correct output files	5 of 5	1 of 5	5 of 5
Rejects bad input	No	Yes	Yes
Time complexity	$O(9^e)$	$O(9^e)$	$O(9^e)$
Solution storage	$O(k)$	$O(1)$	$O(1)$
Classes	One	Two	One
Keeps original code	No	No	Yes

Table 1: Comparison of the GenAI outputs and the final program

6 Selection and Justification (f)

I selected **Mistral AI**. It is the only output that compiles reliably and produces all five correct files, and the program is graded only on the cycle servers. Claude's advantages (input validation, $O(1)$ storage, two classes) are efficiency and robustness gains that are easy to add to Mistral. Claude's problems are correctness failures that would have to be fixed first.

7 Improvements Made (g)

I applied Mistral's design to Christopher's file instead of replacing it, so his code is still visible in GitHub Blame.

Correctness. Added input validation for characters other than `1`–`9` and `_`. Fixed `found_first` starting as `true`, which broke full boards. An invalid board now writes `No solution found.` instead of ending the program. Removed the `--- Solution #n ---` header so output matches the puzzle format.

Execution time. The algorithm is unchanged ($O(9^e)$). I kept the original `isValid` without Claude's extra comparisons, and solutions are written directly instead of being built into one large string.

Space complexity. `printBoard` writes each solution straight to the `ofstream` member, reducing Mistral's $O(k)$ storage to $O(1)$.

Maintainability. Wrapped the code in a `SudokuSolver` class with methods defined outside it, so Christopher's function bodies stay line-for-line. `main` is now a short loop over puzzles 1–5. Every change is marked with a `revised:` comment, and the prologue has "Revised by: Nicolas Villalobos" and lists both GenAIs.